电商销量预测系统 · AI 协作开发教程

版本 V1.0 | 2026-10-13 | 面向：想用 AI 做出一个"能交付的系统"的开发者

本文档是《项目任务书 V1.0.0》§5.2 要求的「AI 协作教程：如何通过 AI 协作实现当前 APP 开发的完整教程」。

⚠ **关于模板的说明**：任务书称「AI 协作教程」为交付物，但**未随任务书提供模板或篇幅要求**（本项目在 PRD 附录 A Q11、开发方案附录 Q11 中已登记为待确认项）。

本文档按「读者能照着复现一遍」的实用目标组织，若负责人后续提供模板，按其调整**结构**，内容无需重写。

目 录

0. 先给结论：这套协作方式有效，但有效的部分不在"提示词" 3

1. 第一段：启动前（占成功率的 50%） 3

1.1 先让 AI 做"文档推导"，不要直接写代码 3

1.2 建立"文档写作规范"（一次建立，全程受益） 4

1.3 冻结"口径"再动手（本项目的三条关键冻结） 5

2. 第二段：开发中（AI 协作的实际形态） 6

2.1 把开发切成"阶段 + 完成判据"，每阶段收口 6

2.2 让 AI 写"会失败的断言"，而不是"会通过的断言" 7

2.3 设计代码结构，让 AI"没法偷懒" 8

2.4 让"验证证据"来自真实环境，而不是代码审查 8

2.5 与本机环境相关的坑，务必写进文档（不是写进脑子） 9

2.6 一份可直接复用的"提示词范式" 10
3. 第三段：收尾（交付物与一致性） 11
3.1 交付物清单要"可核对"，不能"我记得做了" 11
3.2 一致性守卫：文档之间的编号必须能互相验证 12
3.3 交付物的"可用性"要单独验证 13
4. 本项目的真实数据（供估算参考） 13
5. 常见误区（本项目踩过或刻意避开） 14
6. 一页纸回顾 15

0. 先给结论：这套协作方式有效，但有效的部分不在"提示词"

本项目（11 个阶段、35+ 套校验脚本、7 份文档、1 个可运行 Web 应用）**全部由 AI 协作完成**，

过程中产生了 **5 个真实产品缺陷**与 **26 个脚本缺陷** —— 缺陷不是意外，是**必然发生的。**

所以本教程的核心不是「怎么写出好提示词」，而是三件事：

#	关键认知	为什么它比提示词重要
1	**把"验证"从人眼转移到脚本**	AI 会写出"看起来对"的代码。人眼看不出 `header === 1` 而实际有 2 个 `<header>`；脚本一跑就露。
2	**让 AI 无法静默通过**	缺陷最危险的不是报错，是**不报错**（断言恒真、字段被静默丢弃、`except: pass`）。设计上要让它必然暴露。
3	**决策与实现分离**	先冻结"口径"（指标定义、粒度、SLA），再让 AI 写代码。口径一变，前面所有产出作废。

下面的内容按「启动前 → 开发中 → 收尾」三段展开，每段给出**可直接抄的提示词范式**与**真实翻车案例。**

---

1. 第一段：启动前（占成功率的 50%）

1.1 先让 AI 做"文档推导"，不要直接写代码

✕ 低效开场

> 「帮我写一个电商销量预测系统，用 LSTM。」

结果：AI 会给一份"看起来能用"的骨架，但指标口径、预测粒度、SLA 全是它自己编的。

等你发现口径不对时，代码已经写了 3 周。

✅ **有效开场**（本项目实际用法）

> 「这是项目任务书（附件）。
>
> 请你先不要写任何代码。按以下顺序产出**方案文档**：
>
> 1. 逐条解析任务书要求，**标注哪些是明确的、哪些是缺口**（用 ✅/⚠ 区分）；
> 2. 对每个缺口给出**建议默认值 + 理由 + 影响面**，编号为 `Q1`...`Qn`；
> 3. 列出**技术选型的备选与取舍**，说明为什么不选另一个；
> 4. 最后给一份**分阶段执行计划**，每阶段给出「完成判据」（可自动判定的那种）。
>
> 我会逐项确认后你再进入下一个阶段。」

这一步的价值：把"AI 的隐性假设"变成"显式的待确认清单"。

本项目最终收敛出 `Q1~Q18` + `TQ-1~TQ-16` + `QD-01~QD-10` 三层待确认项，

其中用户拍板的 3 条（`Q2` 指标口径 / `Q3` 预测粒度 / `Q6` 训练时机）**直接决定了整个算法层与界面。**

1.2 建立"文档写作规范"（一次建立，全程受益）

本项目的规范（可直接抄）：

规则	作用
优先级 `P0` 阻塞 / `P1` 重要 / `P2` 二期 / `P3` 不排期	避免 AI 把二期需求当阻塞项做，导致范围失控
`P2` 与 `P0-P1` 冲突时无条件让路	消除"都重要"的扯皮
需求必须编号（`FR-模块-序号`），与开发方案的 `M*` 模块一一对应	让"改了 A 忘了 B"变成可被脚本检出

规则	作用
禁止出现「尽量」「可能」「大概」	AI 极爱写这些词。它们让验收无法判定
信息缺失处写 【待补充：xxx】 占位符	显式标记"这里还没定"，而不是悄悄编一个值
标记体系：🔒 硬约束 / 📌 已确认决策 / ⚠ 易错点 / ⏱ 实测数据	让文档自带"哪句不能动"的信息

💡 **为什么"禁止模糊词"这么关键**：一条需求写「尽量在 3 秒内响应」，

AI 会实现成 3.5 秒然后说"符合要求"。写成「🔒 首屏 FCP ≤ 3000 ms（生产构建口径）」，

它就**必须**给出实测值，且口径被写死（不能用 dev 模式糊弄）。

1.3 冻结"口径"再动手（本项目的三条关键冻结）

冻结项	决定	不冻结的后果				
指标口径	主指标 = **销量加权 MAPE**（`Σ\	e\	/Σ\	y\	`）；辅 = 达标序列占比 ≥ 60%；基线 = **移动平均**	用"每序列 MAPE 再平均"会让小序列与大序列权重相同；基线不统一时，提升幅度无从比较
预测粒度	**仓库 × 商品**；仓库列**必须有**，缺失即拒	粒度含糊 → 指标不可比、UI 不知道该展示什么				
训练时机	**纯预训练推理**，不提供在线重训；端到端 ≤ 60 s	若允许在线训练，60 s 预算必然被击穿				

🗞 **答辩可用的说法**：「这三条是立项时就冻结的。冻结之后，所有代码、文档、测试

都朝同一个口径收敛 —— 这也是为什么 11 个阶段做完，指标口径没有漂移过一次。」

2. 第二段：开发中（AI 协作的实际形态）

2.1 把开发切成"阶段 + 完成判据"，每阶段收口

本项目的 11 个阶段（`EP-0` ~ `EP-10`）是这么切的：

```
① EP-0 工程基建    → /health 通 + 前端空布局 + 设计令牌移植
② EP-1 数据库落地 → data/app.db + ORM/DAO
③ EP-2 认证        → 注册登录改密 + 限流 + 权限依赖
④ EP-3 前台首页    → 首页 + 免责声明 + 主题 + 硬指标同源
⑤ EP-4 上传与校验 → 7 步校验 + 5 类错误 + 游客态
⑥ EP-5 预测编排★  → 先脚本后 Web + 状态机 + 批量推理 + 超时软失败
⑦ EP-6 结果可视化★→ 指标口径 + 覆盖率 + 图表 + 导出
⑧ EP-7 记录类      → 历史 / 详情 / 对比 / 工作台
⑨ EP-8 管理端      → 9 接口 + 2 页面 + 全审计 + 上游文档回写
⑩ EP-9 联调测试    → 主干 + 边界 25 条 + 性能 7 项 + 兼容 + 无障碍
⑪ EP-10 文档答辩   → 部署手册 + API 文档 + 教程 + PPT + 演示脚本
```

每个阶段的收口动作固定为 6 步（AI 每阶段结束都跑一遍）：

1. 勾 `DOD`（该阶段的完成定义清单）
2. **跑校验脚本**（不是"我觉得好了"，是真的跑）
3. 留证据（日志落盘到 `outputs/`）
4. 列遗留问题（没解决的**必须登记**，不许悄悄放过）
5. 回写上游文档（改了实现就必须同步文档）
6. 报告进度与下一阶段门槛

⚠ **最重要的经验：阶段"完成"的判据必须是可自动判定的。**

「首页做好了」不可判定；「`/health` 返回 200 且前端空布局可渲染」可判定。

本项目因此写了 35+ 套校验脚本 —— 看起来是额外工作量，实际上它是整个项目**唯一**的质量保证。

2.2 让 AI 写"会失败的断言"，而不是"会通过的断言"

这是本项目最费心、也最有价值的一条。

反例（本项目真实出现过）

`EP-5` 阶段为"结果页尚未实现"写了一条临时断言：页面须标注「EP-6 补全」。

`EP-6` 落地后这条断言**必然失败** —— 它从"守门"变成了"噪音"，

最后被人顺手放宽，于是失去了意义。

同类还有 `verify_ep3_browser.mjs` 里的 `header === 1`：

后来 `DashboardPage` 用了语义化的 `<header>` 标签，DOM 里合计 2 个，断言挂了。

查完发现**页面骨架完好** —— 是断言过时，不是产品缺陷。

正确的做法

原则	做法
断言要有"反例可证伪"	不是"存在某元素"，而是"数量恰为 3，且三个宽度互不相同"
断言过时**必须**升级而非放宽	放宽（`n ≥ 2`）等于自己削弱证据强度，**比不修更糟**
断言失败必须能一眼看出实际值	只有 `assert x == 7` 而没有实际值的断言，失败时等于没给线索
区分"产品缺陷"与"脚本缺陷"	本项目最终统计：产品缺陷 5 个、脚本缺陷 26 个 —— **脚本缺陷是产品的 5 倍**，这很典型

2.3 设计代码结构，让 AI"没法偷懒"

AI 最擅长的偷懒方式有三种，本项目的应对：

偷懒方式	症状	应对（结构性防法）
静默吞异常	`except Exception: pass`，失败时无声无息	清理/删除等关键路径**失败必须打印显式告警**，不许静默
断言恒真	`assert True`、`assert len(x) >= 0`	每条断言必须写出"失败时的实际值"
字段悄悄丢	Pydantic `extra='ignore'` 把响应字段吃掉，前端拿到 `undefined`	契约测试做**双向集合运算**：两端模型字段必须**逐条登记**，任何一侧新增而未登记即失败

💡 本项目最狠的一条守卫：`verify_ep9_contract.py` 用 `MODEL_TO_TS` 映射表强制前后端模型逐条登记。这样「两端其中一个字段悄悄变了」在结构上**不可能静默通过**。

它直接命中过两次真实缺陷：

· `datasetApi.detail()` 泛型写错 → 运行期 `file_name` 全是 `undefined`；

· 管理端新增 `UpdateUserRequest` 覆盖同名模型 → `API-05` 报 `AttributeError` → 500。

2.4 让"验证证据"来自真实环境，而不是代码审查

核心原则：`curl` 返 200 什么都不证明。

本项目把验证分成四层，**每层都真跑**：

层	手段	能抓到什么
算法层	`self_check.py`（69 项）	张量形状、指标口径、分级规则
静态层	AST / 正则扫源码	架构红线（如"`services/` 里零命中训练入口"）、路由注册顺序、硬编码数

层	手段	能抓到什么
		值
接口层	`TestClient` + 临时库	端到端语义、错误码、越权、幂等
真实浏览器	系统 Edge + 原生 CDP	**DOM 是否真的渲染**、控制台错误、网络失败、首屏性能

真实浏览器这一层不可省的真实案例（`EP-10` 实际命中）：

FastAPI 的 `/docs` 默认从 CDN 加载 Swagger UI。国内网络下 CDN 被重置，

症状是「HTML 返 200，但页面白屏」。**用 `curl` 测 100 次都是 200。**

正确的判据是三条一起看：

```
// ① DOM 真的渲染了          → .opblock 数量 > 0（资源没加载时恒为 0）
// ② 引用的资源零外部 http(s) → 扫全部 link[href] / script[src]
// ③ 零加载失败              → CDP Network.loadingFailed 为空、无 4xx/5xx
```

只满足其中一条都不足以排除「HTML 到了、资源没到」。

而浏览器验证还顺手抓到一个**藏在默认参数里**的坑：FastAPI 的 `get_redoc_html()`

默认 favicon 指向 `https://fastapi.tiangolo.com/img/favicon.png`。

只关 `with_google_fonts` 并不能除干净 —— 必然残留一个外链。

这种"藏在默认值里的依赖"，**只有真实浏览器逐一扫资源才能发现。**

2.5 与本机环境相关的坑，务必写进文档（不是写进脑子）

本项目踩过的、**纯属本机环境**的坑（换台机器就重现不了，所以必须文档化）：

坑	症状	防法
系统设了 `HTTP_PROXY=127.0.0.1:xxxxx`	浏览器把 `127.0.0.1` 也塞进代理 → 本地页面全白/502	浏览器启动加 `--no-proxy-server` + `--proxy-bypass-list=<-loopback>`
vite 默认只监听 `[::1]`	代理连不上 → 502	显式 `--host 127.0.0.1`
`vite.config.ts` 代理写死端口	用独立临时库跑测试时，前端能开但**每个接口都失败**，后端日志一条请求都没有	改成读 `VITE_API_TARGET` 环境变量
无头浏览器复用固定 `--user-data-dir` 且只 kill 父进程	子进程持有 profile 锁 → 新实例**静默挂起**：不报错、不产出一张截图、脚本空转 15 分钟	① 每次全新 profile 目录；② 退出时杀**进程树**（`taskkill /T /F`）；③ 给单次截图套超时
`taskkill` 是异步的，紧接着删 profile	文件锁未释放 → 删除静默失败 → **永久残留 1.65 GB**	删除前 `await` 等待 + 重试 + 失败显式告警
平台的删除保护把 `rmSync` 转成异步移入回收站	"删完立刻查"**误判成清理失败**（1 分钟后为 0）	验收清理效果必须**隔一会儿再统计**
Chromium profile 目录是**隐藏属性**	PowerShell `Get-ChildItem` 默认不返回 → 计数偏小 → 误判"删干净了"	统计必须加 `-Force`

⚠ 最后两条是本项目**代价最大**的坑（浪费了一整轮排查）。

教训：**「删除失败」与「删除成功但看不出来」是两种截然不同的故障**，

而它们的表现都是「文件还在」。

2.6 一份可直接复用的"提示词范式"

本项目验证有效的提示词结构（把方括号里换成你的内容）：

```
【背景】[项目名]，[一句话目标]，[已冻结的口径：指标/粒度/SLA]
【当前状态】已完成 [EP-x]，产物在 [路径]，校验脚本在 [路径]
【本次要做】[阶段编号 + 名称]
【硬约束】
  - [不得改动的既有设计]
  - [必须复用的既有脚本/组件]
  - [必须遵守的编号体系]
【完成判据】（可自动判定）
```

```
  1. [脚本名] 退出码 0
  2. [接口] 返回 [具体结构]
  3. [真实浏览器] [具体断言]
【输出要求】
  - 先给方案（文件清单 + 关键设计 + 风险），我确认后再写代码
  - 修改时同步回写 [文档名] 的 [章节]
  - 不与既定口径一致的，必须显式标注并给出替代建议
【纪律】
  - 不许用「尽量/可能/大概」
  - 不许写恒真断言；每条断言必须能打印实际值
  - 关键路径不许静默吞异常
```

三个最有效的字段：

1. **已冻结的口径** —— 防止 AI 每轮重新发明指标定义；
2. **完成判据（可自动判定）** —— 把"做完了"变成"跑得过"；
3. **输出要求：先方案后代码** —— 大幅减少返工（本项目 `EP-5` 做法："**先脚本后 Web**"，即先让算法脱离 Web 跑通并产出评测报告，再接进界面）。

---

3. 第三段：收尾（交付物与一致性）

3.1 交付物清单要"可核对"，不能"我记得做了"

本项目交付物清单的每一项都标注**位置 + 状态 + 生成方式**：

类别	交付物	生成方式
代码	后端 `app/`、前端 `web/`、契约资产 `code/`	——
模型	`outputs/models/lstm_v1.pt` + 报告	`scripts/train_and_register.py`

类别	交付物	生成方式
质量	35+ 套校验脚本	各阶段产出
文档	PRD / UI/UX / 数据库设计 / 开发方案 / 开发技术 / 实施方案 / 测试报告 / 部署运维手册 / API 接口文档 / AI 协作教程 / RAG 语料库 / 项目说明	Markdown 源 → `build_md_docx_formal.py` → docx/pdf
答辩	演示 PPT / 演示脚本 / 冻结演示数据	`make_demo_data.py`
脚本	`verify_*` 全套	——

用脚本核对交付物（本项目 `verify_ep10_docs.py` 的做法）：

把"交付物是否存在、是否非空、关键章节是否齐全"写成断言。

这样"我漏了部署手册"这种事**不可能**蒙混过关。

3.2 一致性守卫：文档之间的编号必须能互相验证

文档一多，最容易出的问题是**交叉引用悬空**：

PRD 里引用 `FR-MOD-05`、技术文档里写 `FR-MOD-05`，但 PRD 里这条被改了编号。

做法：写一个 `verify_doc_refs.py`，扫全部文档里出现的 `FR-*` / `NFR-*` 编号，

断言它们**都真实存在于 PRD**。本项目该项长期保持 **0 悬空**。

同理还有：

- `verify_doc_sql.py`：文档里内嵌的 SQL 与 `schema.sql` **逐行一致**；
- `sync_eval_metrics.py --check`：界面上的硬指标与算法产物 `eval_report.json` **一致**（防"两套数字"）。

> 📌 **"两套数字"是本项目最典型的翻车方式**：模型变好了，界面还显示旧数字，
>
> 于是"算法 14.07%、界面 14.26%"并存，答辩时无法解释。

现在由 QD-06（构建期注入 + 禁止手写）+ GIT-7（重训后必跑同步脚本）+ 断言三者共同守住。

3.3 交付物的"可用性"要单独验证

文档能生成 ≠ 文档能用。本项目在收尾阶段的额外检查：

检查	为什么
docx 的**目录域是否展开**、页码是否正确	Python 生成的 docx 里 TOC 是**未计算的域**，直接用 Word COM 转换 PDF 会得到空白目录
PDF 页数是否符合预期	页数突变通常意味着样式回退（如表格被压成一列）
演示 PPT 是否含**性能实测数据**	DOD-EP10 硬要求；PPT 里写"性能良好"不算
演示数据是否**可复现**	固定随机种子 + 校验脚本断言 README 声明与文件实际一致
离线归档件是否**零外链**	评审方拿到的 HTML 可能在没有网络的环境打开

---

4. 本项目的真实数据（供估算参考）

项	数量
开发阶段	11 个（EP-0 ~ EP-10）
校验脚本	35+ 套，断言总数 **1,000+ 条**
文档	12 份（7 份主体 + 5 份交付辅助）
产品缺陷	**5 个**（F-01~F-05，全部修复并复验）
脚本缺陷	**26 个**（T-01~T-26）
文档勘误	3 处
算法层硬指标	MAPE **14.26%**（要求 ≤ 20%） / MAE↓ **26.42%**（要求 ≥ 15%） / RMSE↓ **27.52%** / 达标序列 **83.95%**（要求 ≥ 60%）

项	数量
端到端性能	**11.85 s**（113 序列，h=14；SLA 60 s，余量 5 倍）；CPU 回退 23.49 s
单次推理	**14–21 ms**
训练	⏱ 45 s（CUDA） / 80 s（CPU）；峰值显存 **111 MB**

💡 **一个反直觉的比例**：脚本缺陷（26）是产品缺陷（5）的 **5 倍**。

这很正常 —— 系统只有一个，但看系统的"眼睛"有很多双，

每多一双眼睛就多一份出错概率。**不要因为脚本失败就断定产品有问题**，

先判断是"产品错了"还是"眼睛看错了"。

---

5. 常见误区（本项目踩过或刻意避开）

误区	现实
「提示词写得够好，AI 就不会写错」	会错。本项目 31 个缺陷里，绝大多数不是"理解错需求"，而是"细节边界没覆盖"
「单元测试过了就等于能交付」	不等于。算法单测全绿时，界面仍可能因契约漂移而显示 `undefined`
「AI 说做完了就是做完了」	必须**独立验证**。本项目多次出现"AI 认为完成、实测未达成"，包括 AI 自己写的校验脚本内部也有 bug
「文档等代码写完再补」	那就补不出来了。本项目把文档写作**揉进每个阶段**（每阶段回写上游文档）
「性能等最后优化」	性能口径要在立项时冻结（本项目：端到端 ≤ 60 s、首屏 ≤ 3 s、预览 ≤ 1 s），否则最后会变成"改需求"
「演示当天再准备」	演示数据必须**冻结 + 预跑**。本项目 `RG-15` 风险项就是为此设的
「把'未测'写成'通过'」	**绝对不要**。本项目四浏览器兼容里 Safari 在 Windows 无发行版，登记为 `BLOCKED`（明确写"未实测，不能据此判定通过"），而不是伪装通过

6. 一页纸回顾

启动前

① 让 AI 先产出方案文档，标注 ✅明确 / ⚠缺口，缺口给建议默认值
② 建立文档规范：编号体系 + 禁模糊词 + 占位符 + 标记体系
③ 冻结口径：指标 / 粒度 / SLA —— 一旦冻结不再讨论

开发中

④ 切成"阶段 + 可自动判定的完成判据"，每阶段 6 步收口
⑤ 让 AI 写"会失败"的断言：反例可证伪、失败能打印实际值、不许恒真
⑥ 结构上防偷懒：关键路径不许静默吞异常、契约必须逐条登记
⑦ 验证分四层，每层真跑；真实浏览器层不可省（curl 200 什么都不证明）
⑧ 环境坑写进文档，不要写进脑子

收尾

⑨ 交付物清单可核对（脚本断言存在 + 非空 + 关键章节）
⑩ 一致性守卫：编号交叉引用、SQL 一致、指标同源
⑪ 验证交付物的"可用性"：目录域展开、页数合理、离线零外链、演示可复现

贯穿始终

⑫ 缺陷会必然发生 —— 把它当成流程的产出而不是意外；
关键是让它在"交付前"暴露，而不是在"答辩现场"暴露

最后一句：本项目最大的体会是 ——

AI 协作的产出质量，取决于你给它设了多少"无法静默通过"的关卡。

提示词决定了它理解得对不对，关卡决定了错了之后会不会被发现。

文档结束。相关文档：《项目开发实施方案 V1.0》（阶段与判据的完整定义）、

《开发技术文档 V1.0》（`BE-*` 约束编号体系）、

《测试报告 V1.0》（5 个产品缺陷与 26 个脚本缺陷的完整记录）。